M2：Python 分析基礎

課程定位： 第三幕 — 用 Python 表達資料操作，建立分析師的最小語法工具集 時數： 3 小時
前置模組： M1 資料思維與 Notebook 工作流 後續模組： M3 NumPy 與陣列思維

模組學習目標

完成本模組後，學生能夠：

1. 使用 Python 基本型態（int / float / str / bool）正確表達資料中的不同欄位類型
2. 選擇適合的容器（list / dict / tuple）來組織一筆或多筆資料記錄
3. 用 `if / elif / else` 寫出單一記錄的分類邏輯，對應資料標記的業務需求
4. 用 `for` 迴圈和 list comprehension 對資料集合進行批量轉換與篩選
5. 把重複的分析步驟定義成函式，讓清理邏輯可以被重複呼叫
6. 使用 `import` 引入外部套件，理解 Python 資料科學生態的基本架構
7. 閱讀簡單的錯誤訊息（TypeError / ValueError / NameError），找出並修正問題

關鍵概念清單

完成模組後，以下每個概念都應能一句話解釋：

- [ ] `int`（整數）：適合計數、ID、數量
- [ ] `float`（浮點數）：適合金額、比率、統計指標
- [ ] `str`（字串）：適合名稱、類別標籤、未轉換的日期
- [ ] `bool`（布林）：True / False，條件判斷的結果
- [ ] 型態轉換：`int()` / `float()` / `str()` 的使用與轉換失敗的處理
- [ ] `list`（列表）：有序、可重複、可修改，適合一欄資料的集合

- [ ] `dict`（字典）：鍵值對，一筆記錄的自然表達，也是 JSON 的直接對映
- [ ] `tuple`（元組）：不可修改的列表，適合固定設定和多值回傳
- [ ] `if / elif / else`：條件分支，決定在什麼條件下執行什麼邏輯
- [ ] `for` 迴圈：逐元素處理，適合需要對集合中每個元素做相同操作的情況
- [ ] list comprehension：一行的轉換與篩選，是 Python 最具代表性的語法模式
- [ ] 函式（function）：把重複工作封裝成可呼叫的流程，輸入 → 處理 → 輸出
- [ ] 預設參數：函式的參數可以有預設值，讓大多數情況不需要傳入所有參數
- [ ] 回傳多值：函式可以同時回傳多個值（以 tuple 形式）
- [ ] `import`：引入外部套件，擴展語言能力
- [ ] 套件別名：`import numpy as np`，是約定俗成的縮寫，不是任意命名
- [ ] TypeError / ValueError / NameError：三種最常見的錯誤類型及其含義

投影片大綱

#	投影片標題	核心訊息	預估時間
S01	只教分析師用得到的 Python	這堂課不是完整 Python 教學，是「資料分析最小工具集」	8 min
S02	數值型態：int 與 float	計數用 int，測量用 float，型態決定能做哪些計算	12 min
S03	字串型態：資料清理的主戰場	str 是最常見的「髒資料來源」，清理操作是核心技能	15 min
S04	容器：list 與 dict	list 是一欄資料，dict 是一筆記錄，兩個容器對應兩種資料視角	15 min

S05	tuple 與型態轉換	tuple 是不可變的 list，型態轉換是清理的基本動作	12 min
S06	條件分支：if / elif / else	資料標記和分類邏輯的基本句型，一個條件對應一個業務規則	15 min
S07	迴圈：for 與 list comprehension	批量處理資料的語言，從 for 到 comprehension 的思維躍升	18 min
S08	函式：把清理步驟變成可重用的工具	函式不是語法練習，是讓分析腳本從「一次性」變成「可重複使用	18 min
S09	錯誤閱讀：把 error 當成線索	TypeError / ValueError / NameError 的模式識別，快速定位問題	12 min
S10	import 與套件生態	Python 的強大 = 語言 + 生態，import 打開套件的大門	10 min
S11	工作坊 A：用 Python 清理一欄髒資料	把 S02-S08 學到的工具組合起來，完成一個真實的清理任務	30 min
S12	工作坊 B：把五個清理步驟封裝成函式庫	把分析流程模組化，體驗腳本可重現的意義	30 min

總計：約 195 分鐘（含工作坊與 Q&A）

詳細投影片內容

S01 — 只教分析師用得到的 Python

核心訊息： 這堂課不是完整的 Python 程式設計課，而是「資料分析師的最小語法工具集」。學的每一個概念，都對應到分析工作的具體場景，沒有場

講師講解要點：

這堂課的邊界： - 會教的： 型態（int / float / str / bool）、容器（list / dict / tuple）、條件（if / elif / else）、迴圈（for）、函式（def）、套件引入（import）。 - 不教的： 類別（class）、繼承、裝飾器、生成器、非同步、型別系統。 - 這些「不教的」不是因為不重要，而是因為資料分析初學階段用不到。等需要用到的時候，你有了足夠的基礎，

學習原則：每個語法概念都對應一個分析場景 - `int` / `float`：訂單數量 vs. 銷售金額的差異 - `str`：清理商品名稱、統一格式 - `list`：一整欄資料的集合 - `dict`：一筆訂單記錄 - `if`：把金額分成「高、中、低」三個標籤 - `for`：對每個商品名稱進行清理 - `def`：把「清理金額欄」這個步驟封裝起來 - `import pandas`：打開資料分析的大門

從 M1 到 M2 的連接： - M1 你學會了「識別問題」：這份資料有缺失值、金額欄是字串、有重複記錄。 - M2 你要學會「執行操作」：用 Python 實際把這些問題修掉。 - 這兩個能力缺一不可：識別了問題但不會修，沒有用；會修但識別不出問題，更危險。

視覺建議：
一個「分析師工具箱」的圖示，工具箱裡有七個工具格，每格一個概念名稱，旁邊是一行對應的分析場景描述。M3 NumPy/pandas 的前置基礎。」

過渡語： 「從最基本的開始。Python 的型態系統告訴我們，資料裡的不同欄位，本質上是不同的東西。」

S02 — 數值型態：int 與 float

核心訊息： 整數用 `int`，帶小數的數字用 `float`。型態決定能做哪些計算，更重要的是，型態決定計算結果的意義。資料清理中，把字串正確轉換為數值型

講師講解要點：

int（整數）的使用場景： - 計數類：訂單數量（3 件商品，不會是 3.5 件）、用戶 ID、頁數。 - 索引類：資料表的列號、排名。 - 注意：Python 的整數沒有大小限制，可以是任意大的整數。

```
order_count = 7
user_id = 10024
rank = 1

# 整數運算
print(10 // 3)  # 3
print(10 % 3)   # 1
print(10 / 3)   # 3.3333...，Python 3 會回傳 float
```

float（浮點數）的使用場景：- 金額：$1,200.50 - 比率：退貨率 0.035（3.5%） - 統計指標：平均值、標準差

```
amount = 1200.50
return_rate = 0.035
average_order = 856.7

# 浮點數的精度問題
print(0.1 + 0.2)          # 0.30000000000000004，不是 0.3
print(round(0.1 + 0.2, 2) # 0.3，用 round() 修正
```

型態對計算的影響：

```
# 整數運算會變成 float
a = 7
b = 2

print(a / b)   # 3.5，float
print(a // b)  # 3，整數
print(a % b)   # 1

# 字串 vs. 數字
print("100" + "200")  # "100200"，字串連接
print(100 + 200)      # 300，數值相加
```

資料分析常見的數值清理問題：

```
# 從資料庫讀進來的金額
raw_amount = "1,200.50"

# 清洗
cleaned = float(raw_amount.replace(",", ""))
print(cleaned)  # 1200.5
print(type(cleaned))  # <class 'float'>
```

視覺建議：三欄對照表：欄一「型態」、欄二「資料場景例子」、欄三「Python 語法與注意事項」。底部加一個「浮點精度陷阱」的小提示框：`0.1 + 0.2 ≠ 0.3`，並說明 `round()` 的解法。

過渡語：

「數字型態還算直觀。資料清理中更麻煩的，是字串型態。資料裡最多髒東西藏在字串裡。」

S03 — 字串型態：資料清理的主戰場

核心訊息：

字串（str）是最常見的「髒資料來源」，空格、大小寫不一致、混入的特殊字元都在字串欄位裡。掌握字串的清

講師講解要點：

字串的建立與基本特性：

```
name = "Alice Wang"
category = "Electronics"
raw_date = "2024-01-15"  # 日期也是字串

# 字串是 immutable
# 一旦建立不可修改
name_clean = name.strip().lower()
print(name)          # "Alice Wang"
print(name_clean)    # "alice wang"
```

最常用的清理操作（對應真實的髒資料問題）：

```
# 從 1 號檔案讀出的 CSV 欄位
raw = "  Electronics  "
print(raw.strip())   # "Electronics"
print(raw.lstrip())  # "Electronics  "
print(raw.rstrip())  # "  Electronics"
```

```
# 從 2 號檔案讀出的客戶名稱
names = ["Alice Wang", "ALICE WANG", "Alice Wang", "alice  wang"]
unified = [n.strip().lower() for n in names]
# 結果：["alice wang", "alice wang", "alice wang", "alice  wang"]
import re
unified = [re.sub(r"\s+", " ", n.strip().lower()) for n in names]
```

```
# 從 3 號檔案讀出的金額欄位
raw_amount = "$1,200.50"
clean_amount = raw_amount.replace("$", "").replace(",", "")
value = float(clean_amount)  # 1200.5
```

```
# [redacted] 4[redacted]
full_name = "Wang, Alice"
last_name, first_name = full_name.split(", ")
print(first_name)  # "Alice"
print(last_name)  # "Wang"
```

字串格式化（輸出分析結果）：

```
# f-string [redacted]Python 3.6+[redacted]
category = "Electronics"
total = 45600.5
count = 230
|
# f-string[redacted]
print(f"{category}: [redacted] ${total:,.2f}[redacted] {count} [redacted]")
# [redacted]Electronics: [redacted] $45,600.50[redacted] 230 [redacted]
|
# [redacted]
:,.2f  → [redacted] + [redacted]
:>10   → [redacted] 10
:.1%   → [redacted]
rate = 0.0354
print(f"[redacted]{rate:.1%}")  # [redacted]3.5%
```

字串的查詢與判斷：

```
email = "alice@company.com"
product_name = "Wireless Headphone 2024"
|
# [redacted]
print("@" in email)                    # True
print(product_name.startswith("Wireless"))  # True
print(product_name.endswith("2024"))        # True
|
# [redacted]
print(product_name.find("Headphone"))       # 9[redacted]
print(product_name.find("Keyboard"))        # -1[redacted] -1[redacted]
```

視覺建議：一個「字串清理操作速查表」，左欄是「問題類型」，右欄是「解決方法 + 程式碼片段」，涵蓋六種最常見的問題。表格下方一行說明：「90% 的字串清理工作，都可以用這六種操作解決。」

過渡語：「單個值（int / float / str）處理好了。真實的資料是一群值的集合。Python 用三種容器來組織集合：list、dict、tuple。」

S04 — 容器：list 與 dict

核心訊息：`list` 是一欄資料（有序的值的集合），`dict` 是一筆記錄（鍵值對，每個鍵對應一個值）。這兩個容器分別對應資料表格的「欄視角」和「列視角」。

講師講解要點：

list（列表）— 欄視角 - 用途：儲存一個欄位的所有值，或一系列需要按順序處理的元素。 - 特性：有順序（index 從 0 開始）、可以有重複值、可以修改。

```
# [illegible]
amounts = [1200.0, 850.5, 3400.0, 125.0, 2100.5]

# [illegible] 0 [illegible]
print(amounts[0])   # 1200.0[illegible]
print(amounts[-1])  # 2100.5[illegible]
print(amounts[1:3])  # [850.5, 3400.0][illegible] 1 [illegible] 2[illegible]

# [illegible]
print(len(amounts))   # 5[illegible]
print(min(amounts))   # 125.0
print(max(amounts))   # 3400.0
print(sum(amounts))   # 7675.0

# [illegible] list
amounts.append(500.0)  # [illegible]
amounts[0] = 1300.0    # [illegible]
print(amounts)
```

```
# [illegible]
categories = ["Electronics", "Clothing", "Electronics", "Food", "Clothing"]

# [illegible] set [illegible] list
unique_cats = list(set(categories))
print(unique_cats)  # [illegible] set [illegible]

# [illegible]
from collections import Counter
print(Counter(categories))  # {'Electronics': 2, 'Clothing': 2, 'Food': 1}
```

dict（字典）— 列視角 - 用途：儲存一筆記錄的所有欄位，鍵是欄名，值是欄位的值。 - 特性：鍵必須唯一、可以修改值、鍵值對的存取比 list 更語意清晰。

```
# [illegible]
order = {
    "order_id": "ORD-0042",
    "customer": "Alice Wang",
```

```
    "amount": 1200.50,
    "category": "Electronics",
    "status": "completed"
}

# 取值
print(order["order_id"])        # "ORD-0042"
print(order.get("amount"))      # 1200.5
print(order.get("discount", 0))  # 0（沒有 KeyError）

# 檢查鍵是否存在
print("status" in order)        # True
print("discount" in order)      # False

# 修改
order["status"] = "shipped"
order["discount"] = 0.1

# 遍歷
for key, value in order.items():
    print(f"{key}: {value}")
```

list of dicts — 多筆記錄的自然表達 - 真實資料通常是多筆記錄，用 list of dicts 來表達。 - 這也是 JSON 陣列格式的直接對映，也是 pandas DataFrame 的底層結構之一。

```
orders = [
    {"order_id": "ORD-001", "amount": 1200.0, "category": "Electronics"},
    {"order_id": "ORD-002", "amount": 350.5, "category": "Clothing"},
    {"order_id": "ORD-003", "amount": 2800.0, "category": "Electronics"},
]

# 取第一筆
print(orders[0])

# ═══ list comprehension S07 ═══
all_amounts = [o["amount"] for o in orders]
print(all_amounts)  # [1200.0, 350.5, 2800.0]
```

視覺建議： 左側一個 list 的視覺化（垂直方塊，每個格子是一個值，旁邊標 index），右側一個 dict 的視覺化（左右兩欄，左欄是鍵，右欄是值，用箭頭連接）。底部加一個「list of dicts」的示意圖，對應一張二維表格（list 是列，dict 是欄）。

過渡語： 「list 和 dict 是最常用的兩個容器。Python 還有第三個容器 tuple，以及在容器之間切換的型態轉換，資料清理會用到。」

S05 — tuple 與型態轉換

核心訊息： `tuple` 是不可修改的 `list`，適合固定設定和函式的多值回傳。型態轉換（`int()` / `float()` / `str()`）是資料清理的基本動作，轉換失敗時要有防護機制。

講師講解要點：

tuple（元組） - 和 list 的差異：tuple 用小括號 `()` 建立，建立後不能修改（immutable）。 - 使用時機一：函式多值回傳（最常見）。 - 使用時機二：不應被修改的設定值，例如顏色的 RGB 值、座標。 - 使用時機三：pandas 的 `df.shape` 回傳 tuple，代表「這個資料結構的形狀是固定的描述，不是可以修改的值」。

```
# tuple ■■■
dimensions = (1000, 5)  # ■■ 1000 ■ 5 ■
colors = ("red", "green", "blue")

# ■■■■ list ■■■■■
print(dimensions[0])  # 1000
print(dimensions[1])  # 5

# ■■■■■■■■■■ tuple ■■■
rows, cols = dimensions
print(f"■■■■■ {rows} ■■■■{cols} ■■■■")

# ■■■■■■■■■■■
# dimensions[0] = 2000  # TypeError: 'tuple' object does not support item assignment
```

```
# ■■■■■■■■■■ tuple■
def get_amount_stats(amounts: list) -> tuple:
    """Return (min, max, mean) of a list of amounts."""
    return min(amounts), max(amounts), sum(amounts) / len(amounts)

amounts = [1200.0, 850.5, 3400.0, 125.0, 2100.5]
lo, hi, avg = get_amount_stats(amounts)
print(f"■■■{lo}■■■■{hi}■■■■{avg:.2f}")
```

型態轉換（Type Conversion）

資料清理最基本的動作：把字串轉成數值型態，或把數值轉成字串。

```
# str → int
count_str = "230"
count = int(count_str)
print(count + 10)  # 240

# str → float
price_str = "1200.50"
price = float(price_str)
print(price * 0.9)  # 1080.45■■■■
```

```
# int / float → str
amount = 1200.5
print("金額" + str(amount))  # "金額1200.5"
# 推薦：f-string
print(f"金額{amount}")
```

型態轉換的失敗處理（非常重要）

```
# 會產生 ValueError
bad_value = "N/A"
float(bad_value)  # ValueError: could not convert string to float: 'N/A'

# 用 try/except 安全轉換
def safe_to_float(value, default=None):
    """
    Try to convert value to float.
    Returns default if conversion fails.
    """
    try:
        return float(str(value).replace(",", "").replace("$", ""))
    except (ValueError, TypeError):
        return default

print(safe_to_float("1,200.50"))  # 1200.5
print(safe_to_float("N/A"))       # None
print(safe_to_float(""))          # None
print(safe_to_float(None))        # None

# 在 pandas 中 errors='coerce' 會這樣
pd.to_numeric(series, errors="coerce")  # 失敗變成 NaN
```

視覺建議：上半部是 tuple vs. list 的對比表格（特性、使用場景、能否修改）。下半部是型態轉換的流程圖：左邊是原始型態（str），中間是轉換（`float()`），右邊是目標型態，下面有一個分叉：「成功 → 回傳結果」和「失敗（ValueError）→ try/except 處理」。

過渡語：「型態和容器都有了。接下來是條件判斷，這是讓 Python 能根據資料值做不同事情的能力。」

S06 — 條件分支：if / elif / else

核心訊息：`if` / `elif` / `else` 是根據資料值做不同處理的基本句型。在資料分析中，最常用於：新增標記欄位（把數值分類成標籤）、資料驗

講師講解要點：

基本語法結構：

```
[redacted]
if [redacted]
    # [redacted]
elif [redacted]
    # [redacted]
else:
    # [redacted]
```

資料分析最常見的用途一：數值標記（Labeling）

```
def label_order_value(amount: float) -> str:
    [redacted]
    Classify an order into value tiers based on amount.
    [redacted]
    if amount >= 5000:
        return "premium"
    elif amount >= 1000:
        return "standard"
    elif amount >= 100:
        return "economy"
    else:
        return "micro"

[redacted]
print(label_order_value(6500))  # "premium"
print(label_order_value(1500))  # "standard"
print(label_order_value(50))    # "micro"
```

資料分析最常見的用途二：資料驗證

```
def is_valid_order(order: dict) -> bool:
    [redacted]
    Check if an order record has valid required fields.
    Returns True if valid, False otherwise.
    [redacted]
    # [redacted]
    if "order_id" not in order:
        return False

    # [redacted]
    if order.get("amount", 0) <= 0:
        return False

    # [redacted]
    valid_statuses = {"pending", "completed", "cancelled", "refunded"}
```

```
    if order.get("status") not in valid_statuses:
        return False

    return True

good_order = {"order_id": "ORD-001", "amount": 1200.0, "status": "completed"}
bad_order  = {"order_id": "ORD-002", "amount": -50.0, "status": "unknown"}
print(is_valid_order(good_order))  # True
print(is_valid_order(bad_order))   # False
```

資料分析最常見的用途三：缺失值的條件處理

```
def get_display_amount(order: dict) -> str:
    """
    Return formatted amount string, or a placeholder if missing.
    """
    amount = order.get("amount")

    if amount is None:
        return "[illegible]"
    elif amount < 0:
        return f"[illegible] ${abs(amount):,2f}"
    else:
        return f"${amount:,,2f}"

print(get_display_amount({"amount": 1200.5})   # "$1,200.50"
print(get_display_amount({"amount": None}))    # "[illegible]"
print(get_display_amount({"amount": -350.0}))  # "[illegible] $350.00"
```

比較運算子快速整理：

```
x = 1200

# [illegible]
print(x > 1000)   # True
print(x >= 1200)  # True[illegible]
print(x == 1200)  # True[illegible]
print(x != 0)     # True[illegible]

# [illegible]
print(x > 100 and x < 5000)  # True[illegible]
print(x < 100 or x > 1000)   # True[illegible]
print(not (x == 0))          # True[illegible]

# [illegible]
value = None
print(value is None)         # True[illegible] None[illegible] is[illegible] ==[illegible]
print(value is not None)     # False
```

視覺建議：左側一個流程圖（菱形 → 兩條路徑，對應 if/else），右側對應三個使用場景的程式碼框（標記 / 驗證 / 缺失值處理），每個場景下方一行「這個函式會在 M3 pandas 中用 `.apply()` 套用到整欄」。

過渡語：「if 處理的是一筆記錄的判斷。當你需要對一群記錄（一整欄、一整個 list）做同樣的事，就需要迴圈。」

S07 — 迴圈：for 與 list comprehension

核心訊息：`for` 迴圈是對集合中每個元素做相同操作的基本句型。list comprehension 是「可讀的一行版本」，是 Python 最具代表性的語法模式，也是分析師最值得掌握的進階寫法。

講師講解要點：

for 迴圈：基本結構

```
for [redacted] in [redacted]:
    # [redacted]
```

```
[redacted]
product_names = [" Wireless Headphone", "USB Cable ", " Laptop Bag "]

cleaned_names = []
for name in product_names:
    cleaned = name.strip().lower()
    cleaned_names.append(cleaned)

print(cleaned_names)
# ['wireless headphone', 'usb cable', 'laptop bag']
```

for 迴圈的常見資料分析用途：

```
[redacted]
import pandas as pd
df = pd.read_csv("sales.csv")

print("=== [redacted] ===")
for col in df.columns:
    missing_count = df[col].isna().sum()
    missing_pct = df[col].isna().mean() * 100
    if missing_count > 0:
        print(f"{col}: {missing_count} [redacted]{missing_pct:.1f}%[redacted]")
```

```
# [redacted]
numeric_cols = ["amount", "quantity", "return_rate"]

for col in numeric_cols:
    lo = df[col].min()
    hi = df[col].max()
    avg = df[col].mean()
    print(f"{col}: [redacted] {lo:.2f}, [redacted] {hi:.2f}, [redacted] {avg:.2f}")
```

enumerate：同時取索引和值

```
categories = ["Electronics", "Clothing", "Food"]

for i, cat in enumerate(categories):
    print(f"{i}: {cat}")
# 0: Electronics
# 1: Clothing
# 2: Food
```

zip：同時迭代兩個 list

```
col_names = ["amount", "quantity", "return_rate"]
thresholds = [0, 1, 0]

for col, threshold in zip(col_names, thresholds):
    invalid = (df[col] < threshold).sum()
    if invalid > 0:
        print(f"{col} [redacted] {invalid} [redacted] {threshold}[redacted]")
```

list comprehension：一行的轉換與篩選

list comprehension 是 Python 最具代表性的語法，把 for 迴圈 + 條件 + append 壓縮成一行，可讀且高效。

```
# [redacted]
[[redacted] for [redacted] in [redacted]]
[[redacted] for [redacted] in [redacted] if [redacted]]
```

```
# [redacted]
product_names = ["  Wireless Headphone", "USB Cable  ", "  Laptop Bag  "]

# [redacted]
# for [redacted]4 [redacted]
cleaned = []
for name in product_names:
    cleaned.append(name.strip().lower())
```

```
# list comprehension 一行
cleaned = [name.strip().lower() for name in product_names]

print(cleaned)
# ['wireless headphone', 'usb cable', 'laptop bag']
```

```
# 轉換 + 篩選
amounts = [1200.0, 350.5, 2800.0, 125.0, 5600.0, 88.0]

# for 版本
high_value = []
for a in amounts:
    if a >= 1000:
        high_value.append(a)

# list comprehension 版本
high_value = [a for a in amounts if a >= 1000]

print(high_value)  # [1200.0, 2800.0, 5600.0]
```

```
# 一組 list of dicts 模擬訂單
orders = [
    {"order_id": "ORD-001", "amount": 1200.0, "category": "Electronics"},
    {"order_id": "ORD-002", "amount": 350.5, "category": "Clothing"},
    {"order_id": "ORD-003", "amount": 2800.0, "category": "Electronics"},
]

# 篩選：只要 Electronics 的金額
electronics_amounts = [o["amount"] for o in orders if o["category"] == "Electronics"]
print(electronics_amounts)  # [1200.0, 2800.0]
```

什麼時候不用 list comprehension： - 邏輯太複雜（超過一個 if 條件，或需要多行處理）→ 用 for 迴圈或函式 - 資料量很大（超過幾萬筆）→ 用 pandas 的向量化操作（M3 會教），效能更好

視覺建議： 中央一個對比框，左側「for 迴圈版本（5 行）」，右側「list comprehension 版本（1 行）」，用雙箭頭標注「等價」。底部加三個語法模式的小卡片：「轉換」、「篩選」、「轉換 + 篩選」，每個附一行程式碼例子。

過渡語：
「你現在可以對一群資料做批量操作了。但如果這些操作需要反覆用在不同的資料集上，每次都要重寫一遍，這

S08 — 函式：把清理步驟變成可重用的工具

核心訊息：

定義函式不是語法練習，而是把「我這次做了什麼」抽象成「任何時候任何人都能呼叫的清理工具」。函式是讓

講師講解要點：

函式的三要素：名稱 / 參數 / 回傳值

```
def [redacted](a1, a2) -> [redacted]:
    """
    Docstring[redacted]
    """
    # [redacted]
    return [redacted]
```

```
[redacted]
def clean_currency_string(raw: str) -> float:
    """
    Convert a currency string like '$1,200.50' or '1200' to float.
    Returns None if the input cannot be converted.

    Args:
        raw: Raw string value from data source

    Returns:
        Float value, or None if conversion fails
    """
    if raw is None:
        return None
    cleaned = str(raw).strip().replace("$", "").replace(",", "")
    try:
        return float(cleaned)
    except ValueError:
        return None

[redacted]
print(clean_currency_string("$1,200.50"))  # 1200.5
print(clean_currency_string("1200"))       # 1200.0
print(clean_currency_string("N/A"))        # None
print(clean_currency_string(None))         # None
print(clean_currency_string(""))           # None
```

預設參數：讓函式更靈活

```
def summarize_column(values: list, top_n: int = 5, show_pct: bool = True) -> None:
    """
    Print a frequency summary of a categorical column.
    """
    from collections import Counter
    counts = Counter(values).most_common(top_n)
```

```
    total = len(values)

    print(f"■ {top_n} ■■■■■ {total} ■■■■")
    for category, count in counts:
        if show_pct:
            print(f"  {category}: {count} ({count/total: 1%})")
        else:
            print(f"  {category}: {count}")

# ■■■■
categories = ["Electronics", "Clothing", "Electronics", "Food", "Clothing", "Electronics"]
summarize_column(categories)              # ■■■■■top 5■■■■■■
summarize_column(categories, top_n=2)     # ■■■ 2 ■
summarize_column(categories, show_pct=False)  # ■■■■■■
```

回傳多個值：用 tuple

```
def assess_column_quality(values: list) -> tuple:
    """
    Assess the quality of a column's values.
    Returns (total_count, missing_count, missing_pct, unique_count)
    """
    import math
    total = len(values)
    missing = sum(1 for v in values if v is None or (isinstance(v, float) and math.isnan(v)))
    unique = len(set(v for v in values if v is not None))

    return total, missing, missing / total * 100, unique

# ■■■■■■■■■■■
total, missing, missing_pct, unique = assess_column_quality([1, 2, None, 3, 2, None])
print(f"■■ {total} ■■■■ {missing} ■■{missing_pct:.1f}%■■■■■ {unique} ■")
```

函式設計原則（對分析師最實用的部分）：- 一個函式做一件事：`clean_and_summarize()` 這種命名通常是設計問題的信號，應該拆成兩個函式。- 函式名稱用動詞開頭：`clean_amount()`、`label_order()`、`validate_record()`，讓呼叫者一眼看懂這個函式做什麼
- 永遠寫 docstring：至少一行說明「這個函式接受什麼，回傳什麼」。- 處理邊界情況：None、空字串、負數、型態錯誤，要想清楚函式應該怎麼處理，不要讓它默默回傳錯誤結果。

視覺建議：
一個「函式三要素」圖：大方框代表函式，左邊是輸入箭頭（標「參數：這次要處理什麼」），右邊是輸出箭頭（標「回傳值」），中間寫函式名稱，下方加一個「好函式 vs. 壞函式」對比：好函式有 docstring、有預設值、有邊界處理；壞函式沒有說明、依賴外部變數、默默吞掉錯誤。

過渡語：「寫函式時很常遇到錯誤。下一張投影片要教你怎麼閱讀錯誤訊息，把 error 當成定位問題的線索，而不是讓你放棄的障礙。」

S09 — 錯誤閱讀：把 error 當成線索

核心訊息：Python 的錯誤訊息不是「程式壞掉了」，而是「程式告訴你哪裡不對」。學會閱讀三種最常見的錯誤（TypeError / ValueError / NameError），能讓你在 30 秒內定位大多數初學者常見的問題。

講師講解要點：

錯誤訊息的結構

```
Traceback (most recent call last):
  File "analysis.py", line 12, in clean_data        ← 哪個檔案
    return float(raw_amount)                        ← 哪一行程式
ValueError: could not convert string to float: 'N/A'  ← 錯誤類型 + 原因
```

閱讀順序：從最底部開始讀，最後一行是錯誤類型和原因，往上是呼叫鏈。

TypeError：對錯誤的型態做了操作

```
# 字串 + 數字
name = "Alice"
count = 3
print(name + count)   # TypeError: can only concatenate str (not "int") to str
print(name + str(count))   # 正確寫法

# 對 None 呼叫方法
value = None
print value.strip()   # AttributeError: 'NoneType' object has no attribute 'strip'
if value is not None:
    value.strip()   # 確保不是 None

# 對不可索引的物件
number = 42
print(number[0])   # TypeError: 'int' object is not subscriptable
```

ValueError：型態對，但值不合法

```
# 字串無法轉成數字
float("N/A")   # ValueError: could not convert string to float: 'N/A'
```

```
int("12.5")      # ValueError: invalid literal for int() with base 10: '12.5'

def safe_float(s):
    try:
        return float(s)
    except ValueError:
        return None  # 轉換失敗回傳 None

# 搭配 pandas 時通常會先檢查
```

NameError：變數不存在

```
# 定義訂單金額
order_amount = 1200.0
print(order_amout)  # NameError: name 'order_amout' is not defined（拼錯了！）

# 嘗試使用計算結果
print(result)  # NameError: name 'result' is not defined
result = compute_something()
print(result)

# Notebook 常常發生：cell 執行順序
# 先跑 Cell 3，再跑 Cell 5，結果變成 Cell 3 報 NameError
```

KeyError：dict 中不存在的鍵

```
order = {"order_id": "ORD-001", "amount": 1200.0}

print(order["status"])  # KeyError: 'status'

# 安全的做法：.get()
status = order.get("status", "unknown")  # 沒有 "status" 鍵，回傳 "unknown"

# 或先檢查
if "status" in order:
    print(order["status"])
```

IndexError：索引超出範圍

```
# 假設 list 只有 3 個
data = [10, 20, 30]
```

```
print(data[5])  # IndexError: list index out of range

# 空 list 的
results = []
print(results[0])  # IndexError: list index out of range

# 安全的做法
if len(results) > 0:
    print(results[0])
```

五種常見錯誤速查表

錯誤類型	觸發原因	典型場景	修正方向
TypeError	對錯誤型態做操作	`str + int`、對 None 呼叫方法	檢查型態，先轉型
ValueError	型態對但值不合法	`float("N/A")`、`int("12.5")`	try/except 包住
NameError	變數不存在	拼錯名、Notebook cell 順序錯	檢查拼寫、重跑 cell
KeyError	dict 鍵不存在	`order["status"]` 但沒有 status	用 `.get()` 加預設值
IndexError	索引超出範圍	空 list 取 `[0]`、超出長度	取值前檢查 `len()`

除錯的系統性方法（給初學者）：1. 讀最後一行：錯誤類型 + 原因 2. 看錯誤指向的那一行程式碼 3. 加 `print()` 把可疑變數的值印出來，確認型態和內容 4. 問：「我以為這個變數是什麼型態？它實際上是什麼型態？」 5. 查速查表：對照上面五種常見錯誤，找到修正方向

```
# 加上 print 檢查
def clean_row(row: dict) -> dict:
    # 印出收到的資料
    print(f"DEBUG: row = {row}")
```

```
    print(f"DEBUG: amount type = {type(row.get('amount'))}")

    amount = row.get("amount")
    # [redacted]...
```

視覺建議：
三個並排的「錯誤卡片」，每張卡片：錯誤類型名稱（大字）、觸發條件（一行）、典型錯誤訊息（程式碼字體

過渡語：「有了型態、容器、條件、迴圈、函式、除錯，最後一個工具是 import。Python 最強大的地方不是語言本身，而是它的套件生態。」

S10 — import 與套件生態

核心訊息：`import` 打開了 Python 套件生態的大門。資料分析 / 機器學習 / AI 的能力都來自套件，不是來自語言本身。理解套件的引入方式，和知道哪些套件解決哪些問題，是分析師的基礎

講師講解要點：

import 的三種寫法與使用時機：

```
import [redacted]
import math
print(math.sqrt(16))    # 4.0
print(math.ceil(2.3))   # [redacted]

[redacted]
# [redacted] math / os / datetime / json [redacted]
```

```
import [redacted] alias [redacted]
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# np / pd / plt [redacted]
# [redacted] Stack Overflow [redacted]
arr = np.array([1, 2, 3, 4, 5])
df = pd.DataFrame({"amount": [1200, 350, 2800]})

[redacted]
# [redacted]
```

```
# [redacted]
from datetime import datetime, timedelta
from pathlib import Path
from collections import Counter, defaultdict

# [redacted]
today = datetime.now()
data_dir = Path("data/")
counts = Counter(["a", "b", "a", "c", "a"])

# [redacted]
# [redacted]
```

資料分析的核心套件地圖（預告 M3-M6）：

套件	用途	何時學
`pandas`	表格資料處理（DataFrame）	M3
`numpy`	數值計算 / 陣列操作	M3
`matplotlib`	基礎視覺化	M4
`seaborn`	統計視覺化（基於 matplotlib）	M4
`scikit-learn`	機器學習	M5/M6
`requests`	HTTP 請求 / API 呼叫	M5

為什麼說「Python 的強大 = 語言 + 生態」：- 純 Python 語法，和其他腳本語言差異不大。- NumPy 讓 Python 能高效處理大規模數值計算（背後是 C 語言）。- pandas 讓 Python 能處理複雜的表格操作，一行頂過去的十行 for 迴圈。- PyTorch / TensorFlow 讓 Python 成為 AI 訓練的主戰場。- 學 Python，不只是學語法，是加入這個生態系。

套件安裝概念（簡短提及）：

```
# [redacted] Notebook [redacted]
pip install pandas numpy matplotlib
```

```
[redacted]
[redacted] Notebook [redacted] ! [redacted]
!pip install pandas
```

視覺建議： 圓心輻射圖。中心是「Python 語言核心」，往外第一圈是資料分析套件（pandas / numpy / matplotlib / seaborn），第二圈是 ML/AI（scikit-learn / PyTorch），第三圈是工程工具（requests / FastAPI）。每個套件名稱下方標一行用途。在 M3-M6 的模組標示對應的套件。

過渡語： 「你現在有了資料分析師的 Python 工具集。是時候把這些工具組合起來，做一個真實的練習。」

S11 — 工作坊 A：用 Python 清理一欄髒資料

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook 資料： `dirty_amounts.py`（清單直接內建在練習中，不需要額外檔案）

目標： 把 S02-S09 學到的型態、字串清理、條件判斷、迴圈、函式組合起來，完成一個從髒資料到乾淨資料的完整清理任務，並把

工作坊 A 任務說明

任務背景：
你收到一份從舊系統匯出的金額欄位，包含各種格式問題。你的任務是寫出一個強健的清理函式，並測試各種邊

Step 0：看清問題所在（5 分鐘）

```
[redacted]
raw_amounts = [
    "$1,200.50",      # [redacted] $ [redacted] + [redacted]
    "856.5",          # [redacted]
    "  3400 ",        # [redacted]
    "N/A",            # [redacted]
    "$125.00",        # [redacted]
    "",               # [redacted]
    "2,100.50",       # [redacted] $ [redacted]
    None,             # [redacted] None
    "-350.00",        # [redacted]
```

```
    "1200abc"        # [redacted]
[redacted]
[redacted]
print(f"[redacted] {len(raw_amounts)} [redacted]")
print("[redacted]")
for i, v in enumerate(raw_amounts):
    print(f"  [{i}] {repr(v)}  (type: {type(v).__name__})")
```

Step 1：寫基本清理函式（8 分鐘）

先寫一個能處理大多數情況的版本：

```
def clean_amount_v1(raw) -> float | None:
    """
    Basic version: clean a single amount value.
    Returns float, or None if cannot be cleaned.
    """
    # [redacted] None
    if raw is None:
        return None

    # [redacted]
    s = str(raw).strip()

    # [redacted]
    if s == "":
        return None

    # [redacted] $ [redacted],
    s = s.replace("$", "").replace(",", "")

    # [redacted]
    try:
        return float(s)
    except ValueError:
        return None

# [redacted] Markdown cell [redacted]
```

Step 2：測試並找出漏洞（5 分鐘）

```
[redacted]
print("=== v1 [redacted] ===")
for raw in raw_amounts:
    result = clean_amount_v1(raw)
    status = "OK" if result is not None else "FAIL"
    print(f"  {repr(raw):20} → {repr(result):12}  [{status}]")
```

在 Markdown cell 說明：哪些測試案例通過了？哪些回傳了 None 但你認為應該有值？

Step 3：改進函式（7 分鐘）

根據測試結果，改進函式：

```
def clean_amount_v2(raw, allow_negative: bool = True) -> float | None:
    """
    Improved version: handles more edge cases.

    Args:
        raw: Raw value to clean (str, int, float, or None)
        allow_negative: If False, negative values are treated as None.

    Returns:
        Float value, or None if cannot be cleaned.
    """
    if raw is None:
        return None

    s = str(raw).strip()
    if not s:
        return None

    # 移除常見符號
    s = s.replace("$", "").replace(",", "")

    try:
        result = float(s)
        if not allow_negative and result < 0:
            return None
        return result
    except ValueError:
        return None

# 測試
print("=== v2 測試 ===")
for raw in raw_amounts:
    result = clean_amount_v2(raw)
    print(f"  {repr(raw):20} → {repr(result)}")
```

Step 4：套用到整個 list（5 分鐘）

```
# 用 list comprehension 處理全部
cleaned_amounts = [clean_amount_v2(v) for v in raw_amounts]

# 統計
valid_amounts = [a for a in cleaned_amounts if a is not None]
failed_count = sum(1 for a in cleaned_amounts if a is None)

print(f"原始：{len(raw_amounts)} 筆")
print(f"成功：{len(valid_amounts)} 筆")
print(f"失敗：{failed_count} 筆")
print(f"最小：{min(valid_amounts):.2f}　最大：{max(valid_amounts):.2f}　總和：{sum(valid_amounts)
```

工作坊 A 核心體驗： 清理函式的開發是「寫 → 測試 → 改進」的迭代過程，不是一次就能寫對的。重要的是：清理函式要能處理所有邊界情況，並且清楚說明「哪些情 None」。

S12 — 工作坊 B：把五個清理步驟封裝成函式庫

類型： 動手工作坊 時間： 30 分鐘 工具： Jupyter Notebook 資料： *`sales_raw.csv`*（課程頁面提供）

目標：
把一份原始資料的清理流程，從「一次性的程式碼」重構成「一組有名字、有說明、可以重用的函式」，體驗模

工作坊 B 任務說明

資料背景： `sales_raw.csv` 包含：訂單 ID（字串）、日期（字串）、商品類別（字串，大小寫不一致）、銷售金額（混合格式字串）、訂單數量（整數 "3.5%"）。

Step 1：寫函式一：資料結構摘要（6 分鐘）

```
import pandas as pd

def describe_dataset(df: pd.DataFrame) -> None:
    """
    Print a quick structural summary of a DataFrame.
    Includes shape, dtypes, and missing value counts.
    """
    print(f"Shape: {df.shape[0]} ■ × {df.shape[1]} ■")
    print(f"\n■■■■■■■:")
    print(df.dtypes.to_string())
    print(f"\n■■■■■:")
    missing = df.isna().sum()
    missing_pct = df.isna().mean() * 100
    for col in df.columns:
        if missing[col] > 0:
            print(f"  {col}: {missing[col]} ■ ({missing_pct[col]:.1f}%)")
        else:
            print(f"  {col}: ■■■")

df = pd.read_csv("sales_raw.csv")
describe_dataset(df)
```

Step 2：寫函式二：金額欄清理（6 分鐘）

```
def clean_amount_column(series: pd.Series) -> pd.Series:
    """
    Clean a currency string column into float.
    Handles formats: '$1,200', '1200.5', '1,000'.
    Values that cannot be converted become NaN.
    """
    return (
        series
        .astype(str)
        .str.strip()
        .str.replace(r"[$,]", "", regex=True)
        .pipe(pd.to_numeric, errors="coerce")
    )

df["amount_clean"] = clean_amount_column(df["amount"])
print("[redacted]")
print(df[["amount", "amount_clean"]].head(10))
print(f"\n[redacted] {df['amount_clean'].isna().sum()} [redacted]")
```

Step 3：寫函式三：類別標準化（5 分鐘）

```
def standardize_categories(series: pd.Series) -> pd.Series:
    """
    Standardize category strings: strip whitespace and lowercase.
    """
    return series.astype(str).str.strip().str.lower()

df["category_clean"] = standardize_categories(df["category"])
print("[redacted]", df["category"].unique())
print("[redacted]", df["category_clean"].unique())
```

Step 4：寫函式四：退貨率解析（5 分鐘）

```
def parse_percentage_column(series: pd.Series) -> pd.Series:
    """
    Parse percentage strings like '3.5%' into float (0.035).
    Returns NaN for values that cannot be parsed.
    """
    return (
        series
        .astype(str)
        .str.strip()
        .str.replace("%", "")
        .pipe(pd.to_numeric, errors="coerce")
        .div(100)
    )

df["return_rate_clean"] = parse_percentage_column(df["return_rate"])
```

```
print(df[["return_rate", "return_rate_clean"]].head())
```

Step 5：寫函式五：完整清理流程（8 分鐘）

把上面四個函式組合成一個「主清理函式」：

```
def clean_sales_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    """
    Apply all cleaning steps to a raw sales DataFrame.
    Returns a new DataFrame with cleaned columns appended.
    Does NOT modify the original DataFrame.
    """
    result = df.copy()

    result["amount_clean"] = clean_amount_column(result["amount"])
    result["category_clean"] = standardize_categories(result["category"])
    result["return_rate_clean"] = parse_percentage_column(result["return_rate"])

    return result

# ■■■■■
df_clean = clean_sales_dataframe(df)

# ■■■■
print("=== ■■■■ ===")
print(f"■■■{len(df)} ■")
print(f"■■■■{len(df_clean)} ■■■■■■■■■■■■■■")
print(f"\n■■■■{[c for c in df_clean.columns if c not in df.columns]}")
```

Step 6：展示與討論（最後 5 分鐘）

講師引導討論： - 如果下週拿到新的銷售資料，這五個函式你還能用嗎？ - 如果業務規則改了（例如把「高價值」的門檻從 1000 改成 2000），需要改幾個地方？ - 函式庫和「把所有程式碼放在同一個 cell」的差異是什麼？

工作坊 B 核心體驗：模組化的清理函式不只是「好看的程式碼」，而是讓你下次遇到同樣資料結構時，只需要呼叫 `clean_sales_dataframe(new_df)` 就能完成清理。這就是 Python 思維的實際體現。

工作坊總結

工作坊	重點技能	預期輸出	核心體驗

工作坊 A	字串清理、型態轉換、邊界處理、函式設計、list comprehension	一個強健的 `clean_amount()` 函式，附帶完整的邊界測試	清理函式的開發是迭代過程，測試邊界
工作坊 B	函式組合、pandas 基本操作、模組化設計	五個可重用的清理函式，加上一個組合這些函式的模組，能讓你換一批新資料，重新跑	

講師注意事項

1. S01 開場 要花足夠時間強調「這堂課的邊界」。學員常常會問「為什麼不教 class？」主動說清楚原因：不是不重要，而是初學階段用不到，等你需要的時候你有足夠的基礎，學起來會
2. S02 浮點精度
通常讓學員意外，這個概念在金融計算中非常重要，值得多花一點時間讓它清楚，但不要深入浮點數的二進
3. S03 字串
是實際工作中最常用的部分，可以讓學員說說自己遇過哪些字串格式問題，用真實場景帶入。
4. S06 條件分支 要多強調「先考慮 None 和邊界情況」的習慣，這是初學者和有經驗者最明顯的差距之一。
5. S07 list comprehension 要反覆強調「等價性」，讓學員不要把它當成魔法，而是「有條件的 for 迴圈的簡寫」。
6. S08 函式 重點是「docstring 和邊界處理」，不是語法本身。花時間在「為什麼要這樣設計」，而不只是「怎麼寫」。
7. S09 錯誤閱讀 可以現場示範：故意寫出一個會報 TypeError 的程式碼，一起閱讀錯誤訊息，找出問題。
8. 工作坊 A 的核心是「測試邊界情況」，不只是「寫出能跑的函式」。確保學員測試了 None、空字串、負數這些邊界。
9. 工作坊 B 中的 pandas 語法（`.str.replace()`、`.pipe()`、`pd.to_numeric()`）學員可能不熟悉。可以說：「這些 pandas 操作的細節 M3 會詳細教，現在先看懂這些函式做什麼就好，不需要記住所有語法。」

課前準備清單（學員）

- [ ] 完成 M1 模組，特別是工作坊 B 的資料稽核報告
- [ ] 確認 Jupyter Notebook 環境可以正常執行
- [ ] 確認已安裝 pandas（`import pandas as pd` 不報錯）
- [ ] 下載範例資料 `sales_raw.csv`（課程頁面提供）

延伸閱讀

參考文獻與引用來源

Python 官方文件

1. Python Built-in Types — 型態系統完整說明。docs.python.org/3/library/stdtypes.html
2. Python Built-in Functions — `int()` / `float()` / `str()` / `len()` / `sum()` 等。docs.python.org/3/library/functions.html
3. PEP 8 — Style Guide for Python Code — 函式命名、docstring 規範。peps.python.org/pep-0008
4. Python 整數無大小限制 — Python 官方文件明確說明：「Integers have unlimited precision.」docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex

浮點數精度

1. IEEE 754 浮點數標準 — `0.1 + 0.2 != 0.3` 是 IEEE 754 二進制浮點數表示法的固有行為，非 Python 獨有。參見 floating-point-gui.de 與 Python 官方說明 docs.python.org/3/tutorial/floatingpoint.html

延伸閱讀

1. Ramalho, L. (2022). Fluent Python, 2nd Edition. O'Reilly Media. — 第一章資料模型，理解 Python 型態系統的設計哲學

2. Real Python. "List Comprehensions in Python" — comprehension 的完整教學，含效能比較。realpython.com/list-comprehension-python